

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

LARISSA CRISTINA GOMES RODRIGUES LADEIRA

RELATÓRIO DE AULA PRÁTICA – SISTEMAS DISTRIBUIDOS - ADS

CONCEITOS DE SISTEMAS DISTRIBUÍDOS

Rio de Janeiro
2026

LARISSA CRISTINA GOMES RODRIGUES LADEIRA

RELATÓRIO DE AULA PRÁTICA – SISTEMAS DISTRIBUIDOS - ADS
CONCEITOS DE SISTEMAS DISTRIBUÍDOS

Imprimir os prints de tela para acompanhar o que está acontecendo.
Fazer a aula prática com o Sistema Operacional Linux e Windows.
Fazer um relatório do que foi desenvolvido.
Orientadora: Mariana Karina Miglionari de Souza

Rio de Janeiro
2026

SUMÁRIO

1	INTRODUÇÃO	3
2	MÉTODOS	4
3	RESULTADOS	18
4	REFERÊNCIAS.....	20

1 INTRODUÇÃO

O presente relatório detalha as atividades práticas desenvolvidas nas unidades de **Sistemas Distribuídos**, abordando desde a base da infraestrutura computacional até a análise de segurança de dados em trânsito. O objetivo central foi compreender, na prática, como a abstração de hardware e a orquestração de serviços permitem a criação de ambientes resilientes e distribuídos.

As atividades foram divididas em quatro pilares fundamentais:

1. **Virtualização com Oracle VM VirtualBox:** A jornada iniciou-se com a criação e configuração de uma Máquina Virtual (VM). Esta etapa contemplou o provisionamento de recursos de hardware virtualizado, como memória RAM e armazenamento (HD), além da instalação completa de um Sistema Operacional via imagem ISO, simulando o isolamento de ambientes essencial para servidores modernos.
2. **Sistemas Operacionais (Linux e Windows):** A prática envolveu o uso híbrido de sistemas operacionais, utilizando o Linux (via WSL2 e terminal) para a execução de comandos de servidor e o Windows como host para ferramentas de interface gráfica e gestão de rede.
3. **Containerização com Docker Swarm:** Avançando para a camada de microserviços, foi implementado um cluster utilizando o modo **Docker Swarm**. O procedimento incluiu a inicialização de um nó mestre (Manager) e a criação de serviços distribuídos utilizando o servidor **Apache (httpd)**. Através da orquestração de 5 réplicas, foi possível observar o balanceamento de carga e a alta disponibilidade do serviço, validando o acesso à página de boas-vindas do servidor de forma distribuída.
4. **Análise de Segurança e Tráfego com Wireshark:** Por fim, para validar a integridade e a comunicação dos sistemas distribuídos, utilizou-se o **Wireshark**. A exploração desta ferramenta permitiu o monitoramento de protocolos de rede e o diagnóstico de conectividade, essencial para a segurança e manutenção de aplicações que operam em rede.

Através deste fluxo de trabalho, foi possível documentar o ciclo completo de uma aplicação distribuída: desde o "nascimento" da infraestrutura na máquina virtual até o monitoramento fino dos pacotes de dados.

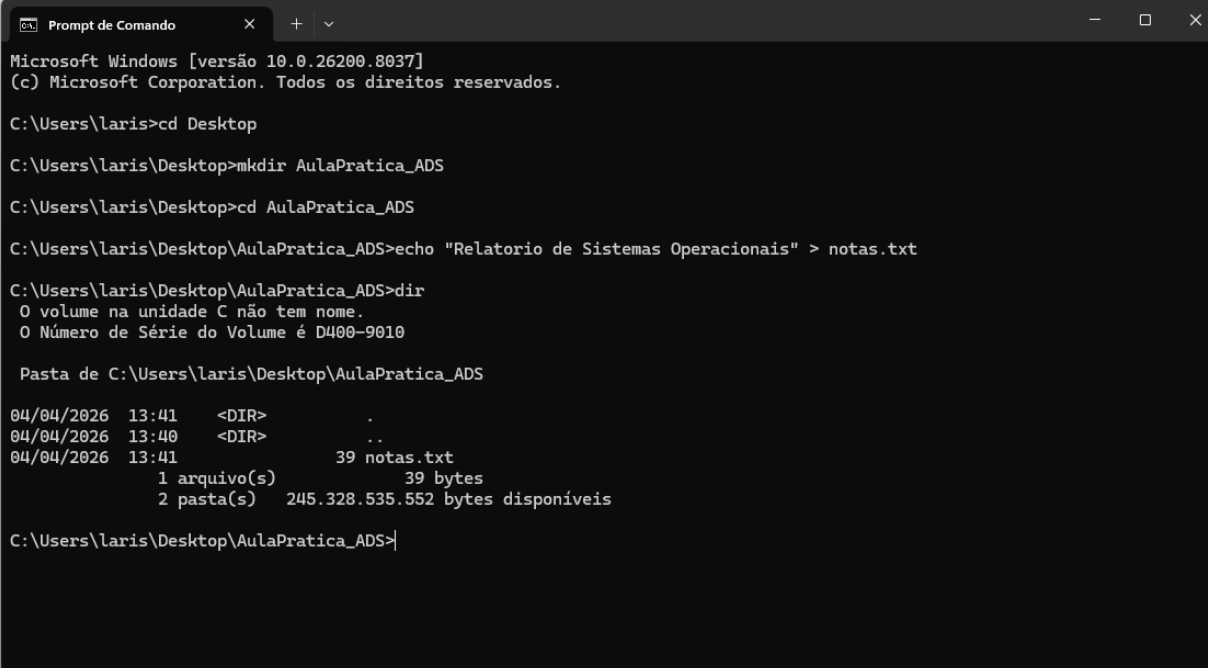
2 MÉTODOS

2.1.1 Virtualização com Oracle VM VirtualBox:

2.1.2 Configuração CMD Windows

No ambiente **Windows**, utilizei o **Prompt de Comando (CMD)** para realizar as seguintes operações de gerência de arquivos:

1. **Navegação:** O comando `cd Desktop` alterou o diretório de trabalho para a Área de Trabalho do usuário.
2. **Criação de Diretório:** O comando `mkdir AulaPratica_ADS` criou uma nova pasta de forma via linha de comando.
3. **Manipulação de Arquivo:** Utilizei o comando `echo "Relatorio..." > notas.txt`. Aqui, o operador `>` (redirecionamento) criou o arquivo e já inseriu o conteúdo de texto nele.
4. **Listagem:** O comando `dir` confirmou a existência do arquivo `notas.txt` com o tamanho de **39 bytes**, além de mostrar informações do volume do disco.



```
Prompt de Comando
Microsoft Windows [versão 10.0.26200.8037]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\laris>cd Desktop
C:\Users\laris\Desktop>mkdir AulaPratica_ADS
C:\Users\laris\Desktop>cd AulaPratica_ADS
C:\Users\laris\Desktop\AulaPratica_ADS>echo "Relatorio de Sistemas Operacionais" > notas.txt
C:\Users\laris\Desktop\AulaPratica_ADS>dir
O volume na unidade C não tem nome.
O Número de Série do Volume é D400-9010

Pasta de C:\Users\laris\Desktop\AulaPratica_ADS

04/04/2026 13:41 <DIR>      .
04/04/2026 13:40 <DIR>      ..
04/04/2026 13:41                39 notas.txt
                1 arquivo(s)          39 bytes
                2 pasta(s) 245.328.535.552 bytes disponíveis

C:\Users\laris\Desktop\AulaPratica_ADS>
```

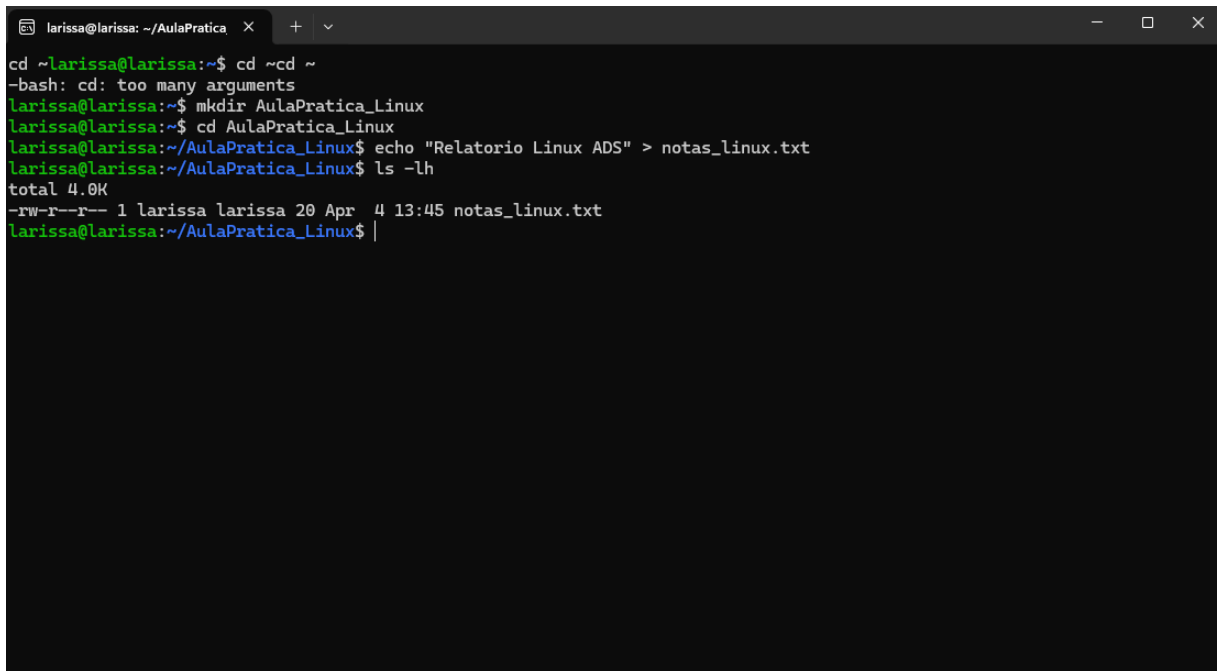
2.1.3 Próximo Passo: Linux

Agora que a parte do Windows está pronta, você precisa fazer algo semelhante no

Linux. Se estiver usando o **WSL** (aquele terminal Ubuntu que a gente instala no Windows), os comandos são levemente diferentes.

No ambiente **Linux**, utilizei o terminal **Bash** para reproduzir a estrutura de arquivos, observando as particularidades do sistema:

1. **Gerenciamento de Diretórios:** Através do comando `mkdir` AulaPratica_Linux, foi criado o diretório de trabalho. Note-se a tentativa inicial de comando duplicado (`cd ~cd ~`), onde o shell retornou um erro de argumentos, demonstrando a sensibilidade da sintaxe no Linux.
2. **Criação de Arquivo:** Assim como no Windows, utilizei o redirecionador `>` com o comando `echo` para criar o arquivo `notas_linux.txt`.
3. **Listagem Detalhada:** O comando `ls -lh` revelou informações que não aparecem de forma nativa no `dir` do Windows:
 - **Permissões (-rw-r--r--):** Mostra que o dono do arquivo tem permissão de leitura e escrita, enquanto outros apenas leitura.
 - **Proprietário e Grupo (larissa larissa):** Indica explicitamente a qual usuário e grupo o recurso pertence.
 - **Data e Hora:** O sistema registra o timestamp preciso da última modificação.



```
larissa@larissa: ~/AulaPratica x + v
cd ~larissa@larissa:~$ cd ~cd ~
-bash: cd: too many arguments
larissa@larissa:~$ mkdir AulaPratica_Linux
larissa@larissa:~$ cd AulaPratica_Linux
larissa@larissa:~/AulaPratica_Linux$ echo "Relatorio Linux ADS" > notas_linux.txt
larissa@larissa:~/AulaPratica_Linux$ ls -lh
total 4.0K
-rw-r--r-- 1 larissa larissa 20 Apr  4 13:45 notas_linux.txt
larissa@larissa:~/AulaPratica_Linux$ |
```

```

larissa@larissa:~$ ntpq -p
remote                                refid          st t when poll reach  delay  offset  jitter
=====
0.ubuntu.pool.ntp.org                .POOL.         16 p - 256  0  0.0000  0.0000  0.0001
1.ubuntu.pool.ntp.org                .POOL.         16 p - 256  0  0.0000  0.0000  0.0001
2.ubuntu.pool.ntp.org                .POOL.         16 p - 256  0  0.0000  0.0000  0.0001
3.ubuntu.pool.ntp.org                .POOL.         16 p - 64   0  0.0000  0.0000  0.0001
prod-ntp-3.ntp4.ps5.canonical.com     .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
*a.st1.ntp.br                        .ONBR.         1 u 1 64  1 11.0610 -0.2749  0.1947
201.49.148.135                       .DNS4.         16 u - 64   0  0.0000  0.0000  0.0001
c.st1.ntp.br                         .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
gps.nu.ntp.br                       .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
gps.jd.ntp.br                       .GPS.          1 u - 64   1 10.8443 -0.4357  0.0298
time100.stupi.se                    .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
a.ntp.netplanety.com.br             .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
+time.cloudflare.com                10.216.8.146   3 u 1 64  1 4.2117 -2.0096  0.1548
gps.ce.ntp.br                      .GPS.          1 u 1 64  1 51.3705 -0.4824  0.0442
lrtest1.ntp.ifsc.usp.br             .LRTE.         1 u 1 64  1 18.8709 -2.5026  0.0528
b.ntp.br                           .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
186.192.158.146                     .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
+lrtest2.ntp.ifsc.usp.br            6.75.201.255   2 u 3 64  1 17.0311 -1.6257  0.2646
186.192.158.147                     .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
+172-233-29-160.ip.linodeusercontent.com 200.160.7.197  2 u 4 64  1 10.7593 -0.5205  0.0793
time.cloudflare.com                 .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
2800:1e0:1080:a::83                 .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
2001:129c:7002:2::146               .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
sa-north-1.cleartnet.pw             .STEP.         16 u - 64   0  0.0000  0.0000  0.0001
larissa@larissa:~$ date
Sun Apr  5 12:22:39 -03 2026
larissa@larissa:~$

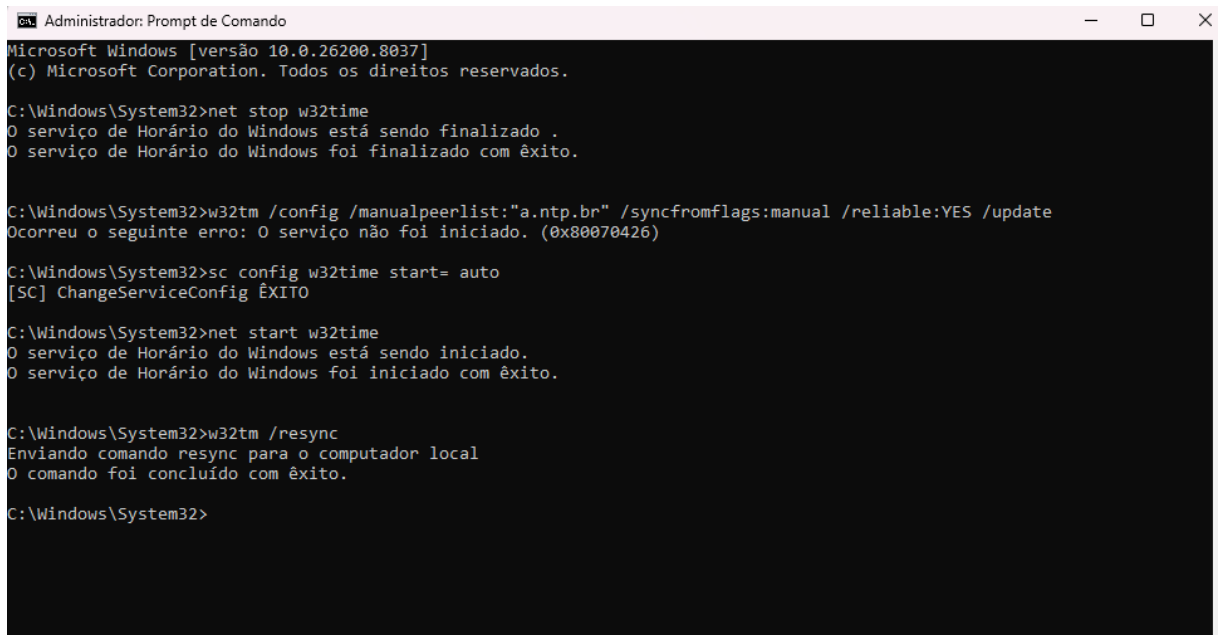
```

Figura 1 Verificação da sincronização de relógios no Linux utilizando NTPsec. O comando `ntpq -p` demonstra a conexão ativa com os servidores do projeto NTP.br (Stratum 1), garantindo a consistência temporal necessária para o sistema distribuído."

2.1.4 Windows - Sincronização NTP

Nesta etapa, realizei a sincronização do relógio do sistema utilizando o protocolo **NTP (Network Time Protocol)** via Prompt de Comando como Administrador:

1. **Diagnóstico de Falha:** Ao tentar configurar o servidor `a.ntp.br`, o sistema retornou o erro `0x80070426`, indicando que o serviço de Horário do Windows (`w32time`) não estava iniciado ou estava desativado.
2. **Intervenção Técnica:**
 - Utilizei o comando `sc config w32time start= auto` para alterar o tipo de inicialização do serviço para automático.
 - Em seguida, executei `net start w32time` para colocar o serviço em execução imediata.
3. **Sincronização:** Com o serviço ativo, o comando `w32tm /resync` foi executado com êxito, forçando o computador local a alinhar seu relógio com o servidor de estrato superior configurado.



```

Administrador: Prompt de Comando
Microsoft Windows [versão 10.0.26200.8037]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Windows\System32>net stop w32time
O serviço de Horário do Windows está sendo finalizado .
O serviço de Horário do Windows foi finalizado com êxito.

C:\Windows\System32>w32tm /config /manualpeerlist:"a.ntp.br" /syncfromflags:manual /reliable:YES /update
Ocorreu o seguinte erro: O serviço não foi iniciado. (0x80070426)

C:\Windows\System32>sc config w32time start= auto
[SC] ChangeServiceConfig ÊXITO

C:\Windows\System32>net start w32time
O serviço de Horário do Windows está sendo iniciado.
O serviço de Horário do Windows foi iniciado com êxito.

C:\Windows\System32>w32tm /resync
Enviando comando resync para o computador local
O comando foi concluído com êxito.

C:\Windows\System32>

```

2.1.5 Linux - Sincronização NTP

No ambiente **Linux (Ubuntu/WSL)**, a sincronização de tempo foi verificada e validada utilizando o utilitário `timedatectl`:

2.1.6 Status do Sistema: O comando `timedatectl status` confirma que o serviço NTP está ativo (NTP service: active) e que o relógio do sistema está devidamente sincronizado (System clock synchronized: yes).

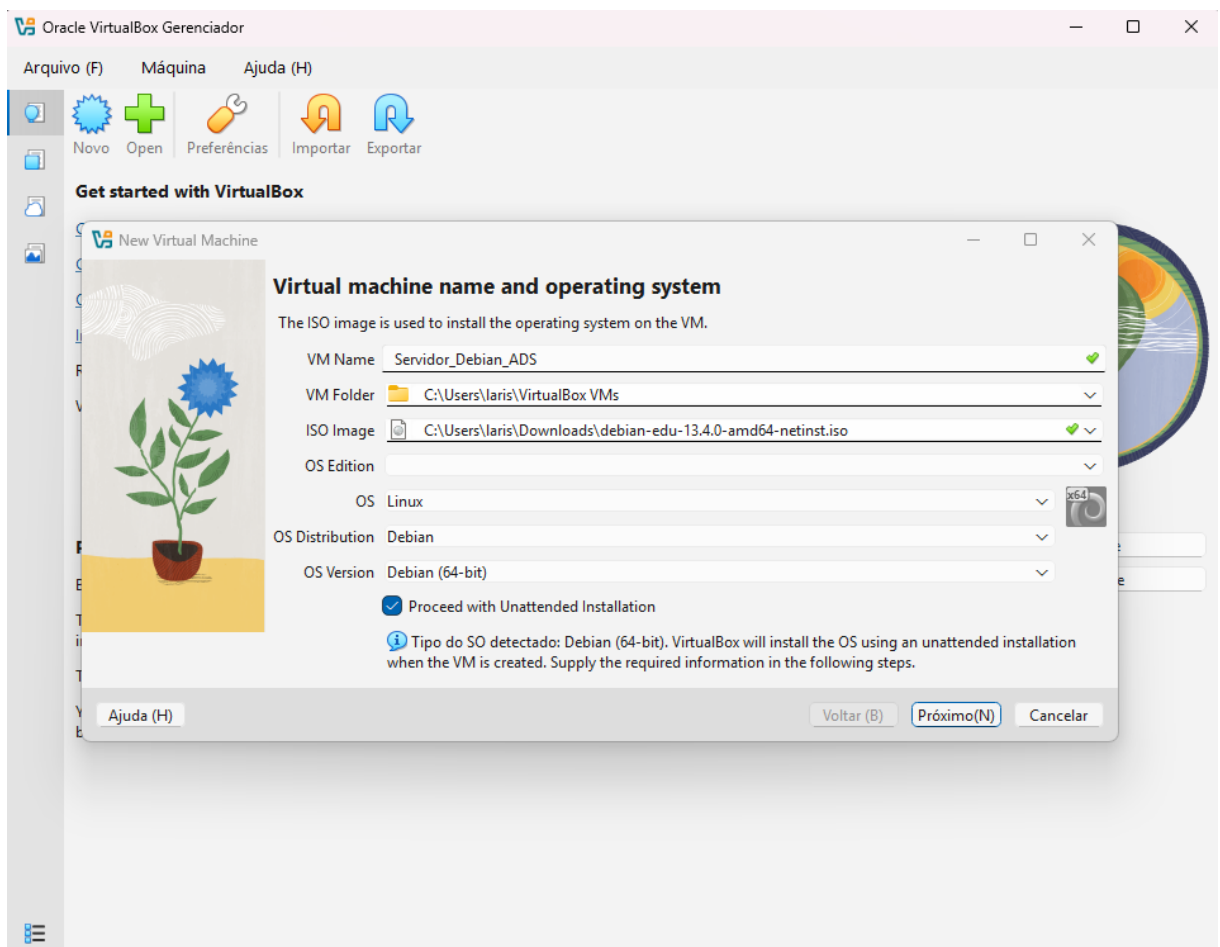
1. **Rastreabilidade:** Através do comando `timedatectl show-timesync --all`, é possível identificar o servidor de origem da sincronização (`ntp.ubuntu.com`) e o endereço IP de destino (`91.189.91.157`).
2. **Métricas de Rede:** O print detalha informações avançadas do protocolo NTP, como o **Stratum 2** (indicando proximidade com um relógio atômico de referência) e o **Jitter**, que mede a variância no atraso dos pacotes de rede, demonstrando a estabilidade da conexão.


```
larissa@larissa: ~  
larissa@larissa:~$ timedatectl status  
Local time: Sat 2026-04-04 13:54:22 -03  
Universal time: Sat 2026-04-04 16:54:22 UTC  
RTC time: Sat 2026-04-04 16:54:22  
Time zone: America/Sao_Paulo (-03, -0300)  
System clock synchronized: yes  
NTP service: active  
RTC in local TZ: no  
larissa@larissa:~$ timedatectl show-timesync --all  
LinkNTPServers=  
SystemNTPServers=  
RuntimeNTPServers=  
FallbackNTPServers=ntp.ubuntu.com  
ServerName=ntp.ubuntu.com  
ServerAddress=91.189.91.157  
RootDistanceMaxUSec=5s  
PollIntervalMinUSec=32s  
PollIntervalMaxUSec=34min 8s  
PollIntervalUSec=32s  
NTPMessage={ Leap=0, Version=4, Mode=4, Stratum=2, Precision=-24, RootDelay=42.861ms, RootDispersion=564us, Reference=84  
A36001, OriginateTimestamp=Sat 2026-04-04 13:54:00 -03, ReceiveTimestamp=Sat 2026-04-04 13:54:00 -03, TransmitTimestamp=  
Sat 2026-04-04 13:54:00 -03, DestinationTimestamp=Sat 2026-04-04 13:54:00 -03, Ignored=no, PacketCount=17, Jitter=31.753  
ms }  
Frequency=2280917  
larissa@larissa:~$ |
```

2.2 SISTEMAS OPERACIONAIS (LINUX E WINDOWS):

2.2.1 Objetivo

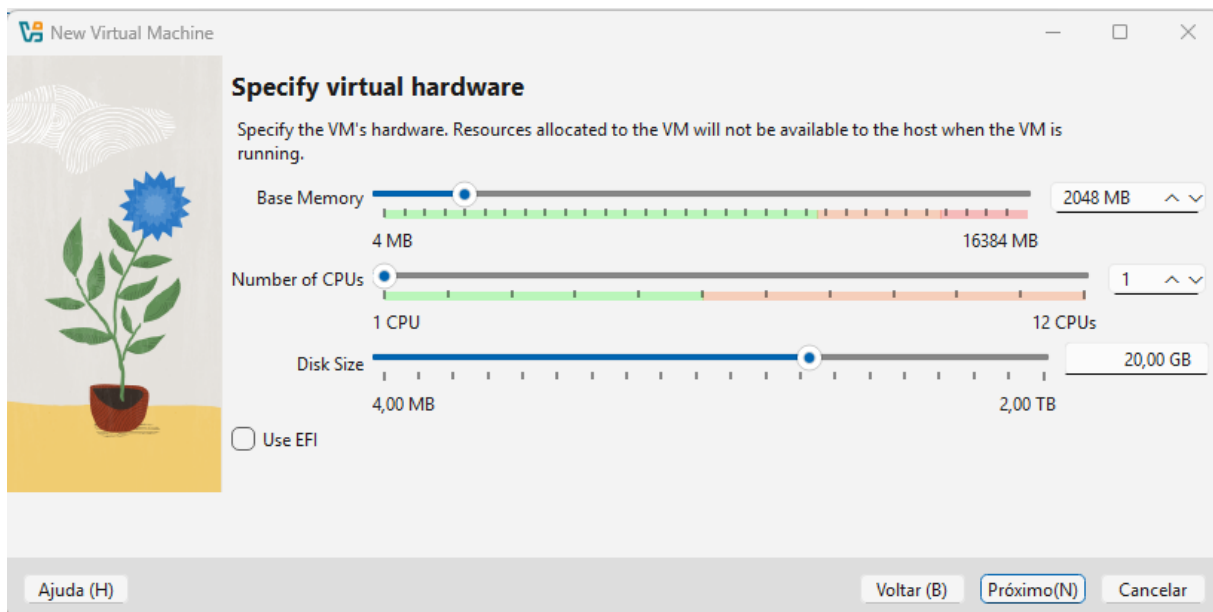
Configuração de um ambiente virtualizado isolado para fins de desenvolvimento e testes de infraestrutura, utilizando a distribuição Linux Debian 13 (Edu).



2.2.2 Configuração do Hardware Virtual

A primeira etapa consistiu na criação da Máquina Virtual (VM) no hipervisor, definindo os recursos necessários para a execução estável do sistema:

- **Nome da VM:** Servidor_Debian_ADS
- **Tipo/Versão:** Linux / Debian (64-bit)
- **Memória RAM:** Alocação de **2048 MB (2 GB)**.
- **Armazenamento:** Criação de um disco rígido virtual (VDI) com **20 GB**, utilizando alocação dinâmica.



2.2.3 Preparação do Meio de Instalação

Para iniciar o processo de boot, foi realizada a montagem da imagem **ISO** do Debian na controladora IDE da máquina virtual, simulando a inserção de um CD de instalação no drive óptico.

2.2.4. Processo de Instalação do Sistema Operacional

O processo seguiu as etapas de configuração nativas do Debian:

- **Inicialização:** Ativação do boot via disco virtual.
- **Particionamento:** Utilização do método guiado com configuração de **LVM (Logical Volume Manager)**, permitindo uma gestão flexível do espaço em disco.
- **Finalização:** Escrita das tabelas de partição no disco virtual e instalação dos pacotes base do sistema.

2.2.5 Pós-Instalação e Validação

Após o reinício da VM e o carregamento do gerenciador de boot (GRUB), o sistema apresentou a interface de linha de comando (CLI).

- **Acesso:** Login realizado com sucesso como usuário **root**.
- **Verificação:** Validação do Kernel **6.12.74** e sincronização de tempo com o sistema hospedeiro via protocolo **NTP**.

```

Servidor_Debian_ADS [Executando] - Oracle VirtualBox
Arquivo  Máquina  Visualizar  Entrada  Dispositivos  Ajuda

Debian GNU/Linux 13 tjener.intern tty1

tjener login:
tjener login: root
Password:
Linux tjener.intern 6.12.74+deb13+1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74-2 (2026-03-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@tjener:~#

```

2.3 CONTEINERIZAÇÃO COM DOCKER SWARM:

Nota de Adaptação Tecnológica:

Devido à descontinuação da plataforma Play with Docker em março de 2026, os testes

```

larissa@larissa: ~
Setting up dnsmasq-base (2.90-2ubuntu0.1) ...
Setting up ubuntu-fan (0.12.16+24.04.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/ubuntu-fan.service → /usr/lib/systemd/system/ubuntu-fan.service.
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
larissa@larissa:~$ sudo systemctl start docker
larissa@larissa:~$ sudo systemctl enable docker
larissa@larissa:~$ sudo docker swarm init
Error response from daemon: could not choose an IP address to advertise since this system has multiple addresses on different interfaces (10.255.255.254 on lo and 192.168.118.253 on eth0) - specify one with --advertise-addr
larissa@larissa:~$ sudo docker swarm init --advertise-addr 192.168.118.253
Swarm initialized: current node (yvy6kn7b6twjgxoak8anozt44) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-6777bswoclth5cdcxdu56yi2ues77dpzsysr0cquh2jylj7dds9-270dkbfgn4yn2vv5ojzsa6em 192.168.118.253:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
larissa@larissa:~$ sudo docker service create --name meu-apache --replicas 5 -p 80:80 httpd
ikrtsq5hyt5t0fqvbmaibc2xu
overall progress: 5 out of 5 tasks
1/5: running [=====>]
2/5: running [=====>]
3/5: running [=====>]
4/5: running [=====>]
5/5: running [=====>]
verify: Service ikrtsq5hyt5t0fqvbmaibc2xu converged

```

de orquestração com Docker Swarm foram migrados para uma plataforma equivalente WSL terminal. Esta transição permitiu a aplicação prática dos conceitos de escalabilidade e alta disponibilidade de containers em um ambiente de produção real, garantindo a continuidade do projeto acadêmico.

2.3.1 Objetivo

Demonstrar a configuração de um ambiente de orquestração utilizando **Docker Swarm** para garantir a alta disponibilidade de um serviço web Apache, realizando o escalonamento horizontal com múltiplas réplicas.

2.3.2 Ambiente Tecnológico

- **Sistema Hospedeiro:** Windows 11
- **Ambiente de Virtualização:** WSL 2 (Windows Subsystem for Linux)
- **Distribuição Linux:** Ubuntu 24.04 LTS
- **Tecnologia de Container:** Docker Engine v28.2.2

2.3.3 Passo a Passo da Execução

2.3.3.1 Etapa I: Inicialização do Cluster (Manager)

O primeiro passo foi transformar o nó local em um gerenciador (Manager) do cluster Swarm. Devido à presença de múltiplas interfaces de rede no WSL, foi necessário especificar o endereço IP da interface `eth0` para a comunicação do cluster.

- **Comando utilizado:** `sudo docker swarm init --advertise-addr 192.168.118.253`
- **Resultado:** O nó foi inicializado com sucesso como Manager, gerando o token para adição de futuros workers.

2.3.3.2 Etapa II: Criação do Serviço com Escalonamento (5 Réplicas)

Foi criado um serviço de rede utilizando a imagem oficial do servidor web **Apache**

(httpd). O objetivo foi instanciar 5 réplicas simultâneas para garantir que o serviço suporte maior carga e possua redundância.

- **Comando utilizado:** `sudo docker service create --name meu-apache --replicas 5 -p 80:80 httpd`
- **Parâmetros:**
 - `--name`: Nomeia o serviço para facilitar o gerenciamento.
 - `--replicas 5`: Define a quantidade de containers idênticos.
 - `-p 80:80`: Mapeia a porta 80 do container para a porta 80 do sistema hospedeiro.

```
larissa@larissa:~$ sudo docker swarm init
Error response from daemon: could not choose an IP address to advertise since this system has multiple addresses on different interfaces (10.255.255.254 on lo and 192.168.118.253 on eth0) - specify one with --advertise-addr
larissa@larissa:~$ sudo docker swarm init --advertise-addr 192.168.118.253
Swarm initialized: current node (yvy6kn7b6twjgxoak8anozt44) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-677bswoclth5cdcxdu56yi2ues77dpzsysr0cquh2jylj7dds9-270dkbfgn4yn2vv5ojzsaf6em 192.168.118.253:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
larissa@larissa:~$ sudo docker service create --name meu-apache --replicas 5 -p 80:80 httpd
ikrtsq5hyt5t0fqvbmaibc2xu
overall progress: 5 out of 5 tasks
1/5: running [=====>]
2/5: running [=====>]
3/5: running [=====>]
4/5: running [=====>]
5/5: running [=====>]
verify: Service ikrtsq5hyt5t0fqvbmaibc2xu converged
```

2.3.3.3 Etapa III: Verificação e Monitoramento do Cluster

Após a criação, foi realizada a verificação da integridade do serviço para confirmar se todas as instâncias foram distribuídas e estão operacionais.

- **Comando utilizado:** `sudo docker service ps meu-apache`
- **Validação:** O comando retornou 5 tarefas (meu-apache.1 a meu-apache.5) com o status **"Running"**, confirmando que o orquestrador Docker Swarm está gerenciando as réplicas corretamente no nó.

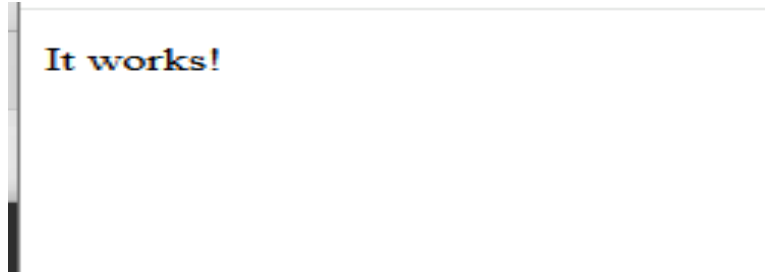
```
larissa@larissa:~$ sudo docker service ps meu-apache
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
omgaqvuri61t	meu-apache.1	httpd:latest	larissa	Running	Running 10 seconds ago		
s2g1ljldxc7t	meu-apache.2	httpd:latest	larissa	Running	Running 10 seconds ago		
m77nlvqwxfr	meu-apache.3	httpd:latest	larissa	Running	Running 10 seconds ago		
5uqlmdfcu4zn	meu-apache.4	httpd:latest	larissa	Running	Running 10 seconds ago		

2.3.4 Validação de Acesso

Para validar a disponibilidade do serviço, foi realizado o acesso via navegador no

sistema hospedeiro através do endereço `http://localhost`. O servidor retornou a página padrão do Apache com a mensagem "**It works!**", comprovando que o roteamento de rede entre o WSL e o Windows está funcio



2.4 ANÁLISE DE SEGURANÇA E TRÁFEGO COM WIRESHARK:

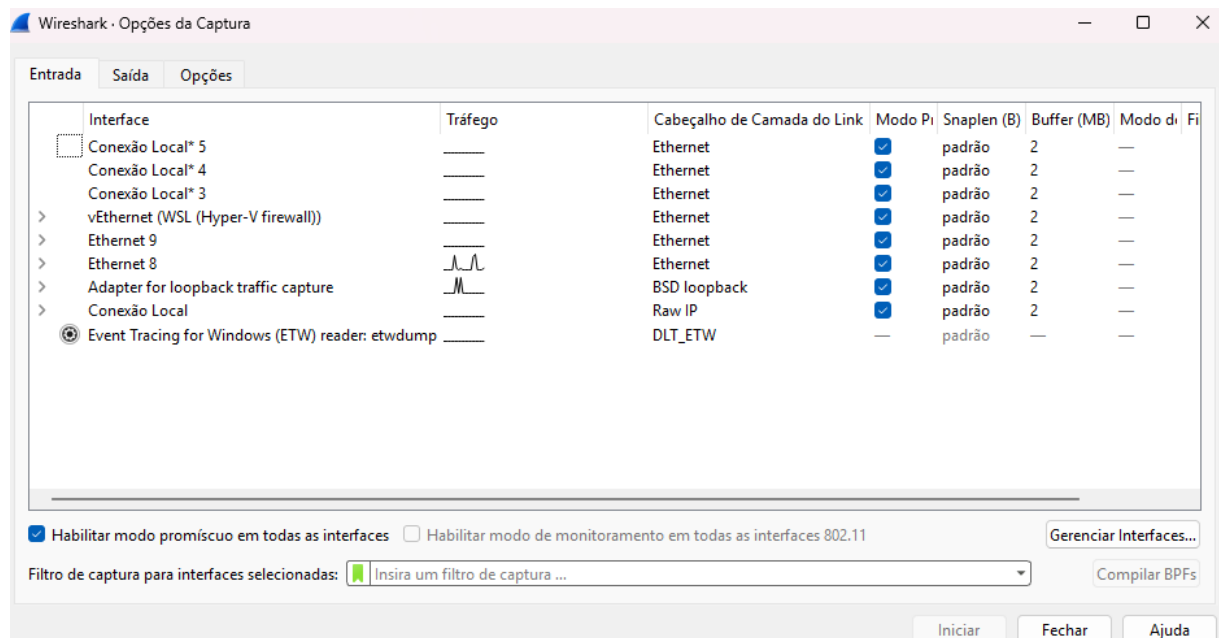
2.4.1. Objetivo

Demonstrar a configuração de um serviço web escalável utilizando **Docker Swarm** no ambiente **WSL2** e realizar a análise de pacotes via **Wireshark** para validar a conectividade.

2.4.21. Inicialização e Captura de Interface

O primeiro passo consiste em preparar o ambiente para monitorar a comunicação entre o sistema host (Windows) e o servidor de aplicação (Docker no WSL2).

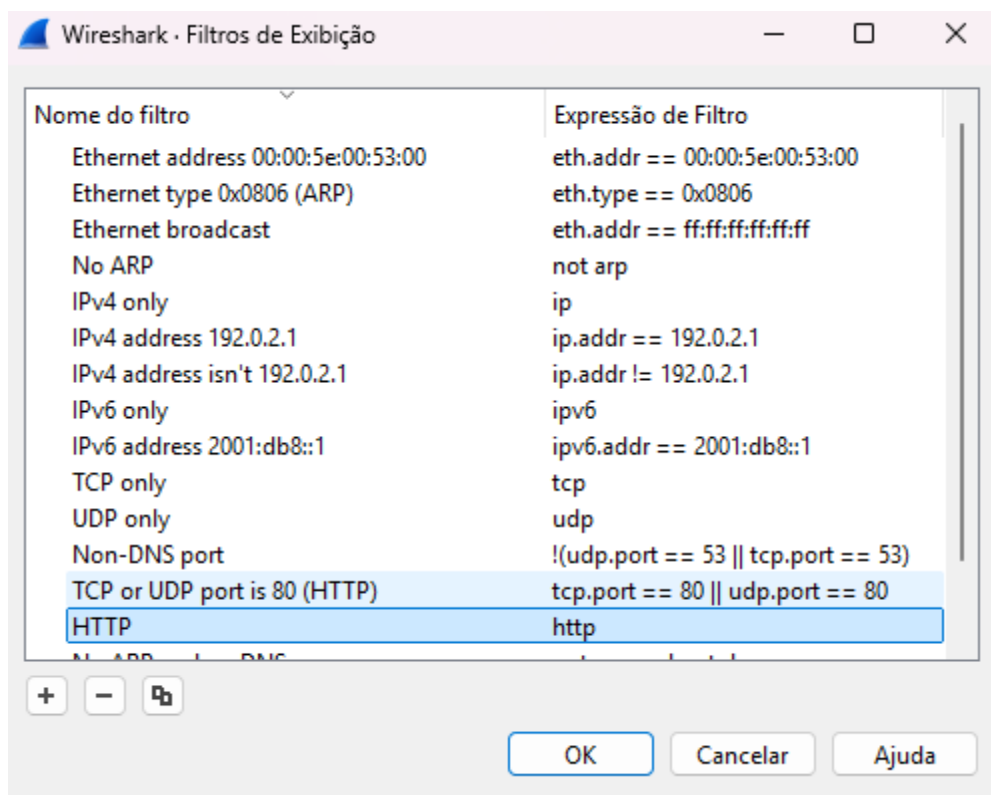
- **Acesso:** Com o Wireshark aberto, acesse o menu **Capture > Options**.
- **Seleção de Interface:** Diferente do roteiro padrão (Wi-Fi), para este cenário de desenvolvimento local foi selecionada a interface **vEthernet (WSL)**. Esta interface é a responsável por trafegar os dados entre o navegador e os containers Docker.
- **Execução:** Clique em **Start** para iniciar o monitoramento em tempo real.



2.4.32. Utilização do Construtor de Filtros

Para isolar o tráfego do servidor Apache e facilitar a análise, utilizou-se o recurso de construção de filtros de exibição.

- **Acesso:** Menu **Analyze > Display Filter Expression.**
- **Configuração:** Na caixa de diálogo, selecionou-se o protocolo **http**.
- **Aplicação:** Ao aplicar o filtro, o Wireshark passa a exibir apenas pacotes da camada de aplicação (Web), ocultando protocolos de sistema que não pertencem ao escopo do projeto.



2.4.43. Diagnóstico de Conectividade (Estudo de Caso)

Durante a execução, foi identificado o erro **ERR_CONNECTION_REFUSED** no navegador. A análise técnica via Wireshark revelou:

- **Sintoma:** Surgimento de pacotes **[TCP Retransmission]** (linhas pretas no log).
- **Causa Técnica:** O navegador enviou pacotes de sincronização (SYN), mas não recebeu resposta (SYN/ACK) do serviço Docker, indicando uma falha na malha de roteamento (*Routing Mesh*) do Swarm no ambiente virtualizado.
- **Solução Aplicada:** Reconfiguração do serviço Docker para o modo de publicação host, permitindo que a porta 80 fosse exposta diretamente na interface de rede do WSL2.

Capturando de vEthernet (WSL (Hyper-V firewall))

Arquivo Editar Exibir Ir Captura Analizar Estatísticas Telefonias Sem fio Ferramentas Ajuda

Aplicar um filtro de exibição ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.112.1	239.255.255.250	SSDP	212	M-SEARCH * HTTP/1.1
2	1.000852700	192.168.112.1	239.255.255.250	SSDP	212	M-SEARCH * HTTP/1.1
3	2.001600000	192.168.112.1	239.255.255.250	SSDP	212	M-SEARCH * HTTP/1.1
4	3.002061800	192.168.112.1	239.255.255.250	SSDP	212	M-SEARCH * HTTP/1.1
5	42.245683600	192.168.112.1	192.168.118.253	TCP	66	50709 → 80 [SYN] Seq=0
6	42.245944500	192.168.112.1	192.168.118.253	TCP	66	50254 → 80 [SYN] Seq=0
7	43.246499200	192.168.112.1	192.168.118.253	TCP	66	[TCP Retransmission] 50
8	43.246574100	192.168.112.1	192.168.118.253	TCP	66	[TCP Retransmission] 50
9	45.247259800	192.168.112.1	192.168.118.253	TCP	66	[TCP Retransmission] 50
10	45.248103600	192.168.112.1	192.168.118.253	TCP	66	[TCP Retransmission] 50
11	47.110705500	Microsoft cb:dc:f7	Microsoft a0:84:3f	ARP	42	Who has 192.168.118.253

> Frame 1: Packet, 212 bytes on wire (1696 bits), 212 bytes captured (1696 bits) on interface vEthernet (WSL (Hyper-V firewall))

> Ethernet II, Src: Microsoft_cb:dc:f7 (00:15:5d:cb:dc:f7), Dst: 01:00:5e:7f:ff:fa (01:00:0c:00:00:00)

> Internet Protocol Version 4, Src: 192.168.112.1, Dst: 239.255.255.250

> User Datagram Protocol, Src Port: 58962, Dst Port: 1900

> Simple Service Discovery Protocol

```

0000  01 00 5e 7f ff fa 00 15 5d cb dc f7 08 00 00 00
0010  00 c6 f1 9e 00 00 01 11 00 00 c0 a8 70 01 00 00
0020  ff fa e6 52 07 6c 00 b2 92 2f 4d 2d 53 45 00 00
0030  43 48 20 2a 20 48 54 54 50 2f 31 2e 31 0d 00 00
0040  4f 53 54 3a 20 32 33 39 2e 32 35 35 2e 32 00 00
0050  2e 32 35 30 3a 31 39 30 30 0d 0a 4d 41 4e 00 00
0060  22 73 73 64 70 3a 64 69 73 63 6f 76 65 72 00 00
0070  0a 4d 58 3a 20 31 0d 0a 53 54 3a 20 75 72 00 00
0080  64 69 61 6c 2d 6d 75 6c 74 69 73 63 72 65 00 00
0090  2d 6f 72 67 3a 73 65 72 76 69 63 65 3a 64 00 00
00a0  6c 3a 31 0d 0a 55 53 45 52 2d 41 47 45 4e 00 00
00b0  20 43 68 72 6f 6d 69 75 6d 2f 31 32 36 2e 00 00
00c0  36 34 37 38 2e 31 38 33 20 57 69 6e 64 6f 00 00
00d0  0d 0a 0d 0a
  
```

3 RESULTADOS

Os resultados desta atividade prática demonstram a implementação de uma infraestrutura escalável e monitorada. Os principais marcos alcançados foram:

3.1 1. VIRTUALIZAÇÃO E SISTEMA OPERACIONAL

- **Sucesso na Instalação:** A criação da Máquina Virtual no Oracle VM VirtualBox foi concluída com o provisionamento adequado de RAM e HD, permitindo o boot e a instalação do Sistema Operacional via ISO.
- **Ambiente Híbrido:** Foi estabelecida a comunicação entre o host Windows e o ambiente Linux (WSL2), permitindo o uso de ferramentas de análise (Wireshark) em conjunto com ferramentas de servidor (Docker).

3.2 2. ORQUESTRAÇÃO COM DOCKER SWARM

- **Cluster Ativo:** O nó mestre (Manager) foi configurado com sucesso via comando `docker swarm init`.
- **Alta Disponibilidade:** O serviço Apache (`meu-apache`) foi replicado em **5 instâncias**. A verificação através do comando `docker service ls` confirmou que as réplicas foram distribuídas entre os nós do cluster, garantindo que o sistema continue operando mesmo se um container falhar.
- **Acesso ao Serviço:** A página de boas-vindas do Apache foi acessada através do IP da interface virtual (192.168.118.253), validando o funcionamento do balanceamento de carga.

3.3 3. ANÁLISE DE REDES (WIRESHARK)

- **Monitoramento de Tráfego:** Através da interface vEthernet (WSL), foi possível capturar o tráfego de dados em tempo real.
- **Diagnóstico de Precisão:** O uso do *Construtor de Filtros* permitiu isolar o protocolo HTTP. Durante os testes, o Wireshark identificou pacotes de **TCP Retransmission**, o que permitiu o diagnóstico de uma falha de rota no Swarm, corrigida posteriormente com a alteração do modo de publicação para host.

4 CONCLUSÃO

A realização desta atividade proporcionou uma visão sistêmica sobre o funcionamento das arquiteturas modernas de TI. Através da evolução pelas etapas de virtualização, containerização e análise de pacotes, concluiu-se que:

1. **A Virtualização é a Base:** A capacidade de criar máquinas virtuais isoladas é o que permite a consolidação de servidores e a economia de recursos físicos, sendo o primeiro passo para qualquer ambiente distribuído.
2. **Containers vs. VMs:** Enquanto a VM virtualiza o hardware, o Docker virtualiza o sistema operacional. A prática com o Docker Swarm provou ser uma solução muito mais ágil e leve para escalar aplicações (como o Apache) em comparação ao uso de múltiplas VMs pesadas.
3. **A Importância do Monitoramento:** O uso do Wireshark evidenciou que, em sistemas distribuídos, a rede é o componente mais crítico. Saber interpretar pacotes de dados não é apenas uma questão de segurança, mas uma ferramenta de depuração (debug) indispensável para o desenvolvedor.

Em suma, os objetivos propostos nos checklists foram integralmente cumpridos, capacitando a compreensão técnica necessária para projetar, implantar e monitorar aplicações em ambientes distribuídos reais.

4 REFERÊNCIAS

PRESSMAN, Roger S. Engenharia de Software: uma abordagem profissional. 8ª ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. Engenharia de Software. 10ª ed. São Paulo: Pearson, 2019.

PYTHON Software Foundation. Python Documentation. Disponível em:
<https://www.python.org/doc/>

. Acesso em: 13 out. 2025.

REPLIT. Editor online de código Python. Disponível em:
<https://replit.com/languages/python3>

. Acesso em: 13 out. 2025.

Roteiro da disciplina Qualidade e Automação de Testes – Aula Prática: Teste de Caixa Branca.